Confidence-Aware Adaptive Verification of LLM Recommendations in Introductory Programming Tasks

Lidia Davydova

Supervisor: Paolo Ciancarini

September 28, 2026

1 Introduction and Research Problem

1.1 Context and Motivation

Large language models (LLMs) are increasingly used in programming education and software engineering to generate, explain, and debug code and to propose answers to tasks [1], [2]. Because such systems can produce plausible but incorrect outputs, the quality of the joint human–AI outcome depends not only on model accuracy but on *appropriate* reliance: accepting correct recommendations and rejecting incorrect ones; over-reliance and under-reliance are the corresponding failures [3], [4]. Verification support can help users evaluate recommendations, and programming tasks suit it well: correctness can often be checked independently through execution, tests, traces, or counterexamples—evidence grounded in the task itself rather than in another LLM-generated explanation. However, verification on every trial adds unnecessary interaction cost, and simply providing more information (e.g., AI explanations) does not automatically reduce over-reliance [4], [5]; explanations help only if users can interpret them and connect them to the recommendation [6]. This motivates verification evidence that is independent of the LLM and shown *selectively*.

1.2 Research Problem

The research problem is how to provide verification support *selectively*, using only information available before the correctness of the LLM recommendation is known. The thesis investigates whether a simple adaptive policy, based primarily on the user's initial confidence, can selectively provide independent verification support while maintaining appropriate reliance. The focus is a simple, inter-

pretable policy—not a new LLM, a new verification algorithm, or an optimisation method.

2 Research Question

How does confidence-aware adaptive verification affect reliance on LLM recommendations in introductory programming tasks compared with static and no verification? The question decomposes as follows.

RQ1. How do no, static, and adaptive verification affect final answer accuracy, over-reliance, and under-reliance?

RQ2. Can adaptive verification achieve reliance and accuracy outcomes comparable to static verification at lower verification cost?

RQ3. How do the effectiveness and cost of the adaptive policy differ across initial confidence levels and task types?

In addition, a policy premise check examines whether initial confidence is associated with initial answer correctness (Section 5.3).

3 Novelty and Contributions

The novelty is *not* the observation that confidence relates to reliance on AI—this is well documented [7] and taken here as a premise—but the combination of three elements usually studied in isolation: (i) confidence as a per-trial *trigger* deciding *when* support is shown, rather than an outcome to be calibrated or an AI-side uncertainty signal; (ii) *independent, execution-based* verification evidence (tests, traces, counterexamples) rather than additional AI-generated explanations or confidence scores; (iii) a programming setting in which recommendations are

pre-validated and evidence is objective. The cost–quality trade-off—comparable outcomes with fewer interventions—is evaluated explicitly, treating verification cost as a first-class evaluation dimension [7], [8].

The thesis is expected to contribute: (1) empirical evidence on whether a simple confidence-based policy matches static verification while displaying support on fewer trials (RQ1–RQ3); (2) a validated benchmark of 18 introductory Python tasks with fixed, pre-validated recommendations, task-type and difficulty labels, and verification data; (3) a reproducible prototype of the three conditions with complete trial logging; and (4) an analysis pipeline computing accuracy, over-, under-, and appropriate reliance, verification rate, decision changes, and verification efficiency. The contribution is deliberately not a new LLM, algorithm, or tutoring system, but a controlled, reproducible human–AI interaction experiment.

4 State-of-the-Art

Appropriate reliance and verification cost. The central problem in human–AI decision-making is not whether users trust AI but whether they rely on it appropriately: over- and under-reliance arise when users fail to calibrate reliance to recommendation quality [3], [4]. Cognitive forcing—slowing down, deliberating, or inspecting evidence before accepting advice—reduces inappropriate reliance [5], but explanations alone do not when they are not diagnostic or independent of the recommendation [4]; verification support is therefore a promising intervention. Moreover, correctness alone is insufficient for evaluating AI: verification consumes time and effort, so an accurate system whose errors are hard to detect may be less useful than a slightly less accurate one whose errors are easy to detect

[8]. The thesis operationalises this view: verification cost is proxied by display frequency, response time, and decision changes, and the adaptive policy is judged by whether it preserves appropriate reliance at lower cost.

Self-confidence calibration and adaptive support. Closest to this thesis, [7] show that discrepancies between initial self-confidence and actual correctness are systematically associated with inappropriate reliance and propose *calibrating* confidence; related work adapts support to user trust [9]. Here, by contrast, confidence is an *input signal*—a lightweight trigger deciding *when* support is shown—not an outcome to be corrected on every trial; the support is independent, execution-based evidence, not AI-generated scores or explanations; and the tasks provide ground-truth evidence at decision time, unlike estimation tasks. Their finding is taken as a premise: if confidence is diagnostic, it can also decide *when* verification is worth its interaction cost—a cost dimension absent from calibration-oriented work and a primary outcome here (RQ2).

LLM assistance in programming and research gap. LLMs are increasingly used for code generation, debugging, and explanation [1], [2] but can be plausible yet incorrect, so users must evaluate recommendations critically. Prior work shows that confidence, trust calibration, and verification cues influence reliance, with the effectiveness of interventions depending on the task, the interface, and the cost of engagement [4]. What remains unknown is whether a participant's initial confidence can serve as a lightweight trigger for selectively displaying *independent verification evidence* after an LLM recommendation in introductory programming tasks—the gap addressed here.

5 Goal of the Thesis

The goal is to design, implement, and empirically evaluate an experimental system that provides *selective* verification support for LLM recommendations in introductory programming tasks, and to determine whether a simple confidence-based adaptive policy can allocate this support more economically than an always-on (static) policy while maintaining appropriate reliance. The schedule is given in Section 6.

5.1 Task Domain, Recommendations, and Response Space

The benchmark comprises 18 introductory Python tasks of four types—*predict output*, *identify bug*, *check against specification*, and *select correct implementation*; "write or repair code" tasks are excluded because they lack a closed response space. The LLM recommendation is conventional, pre-generated, and validated in advance; its correctness is hidden from the participant and the policy, so differences between conditions are attributable to the availability of verification support, and recommendation effects can be separated from verification effects.

Every task has a *closed-form response space*, which makes correctness and change detection well-defined: a canonical output value compared after fixed normalisation rules (predict output); a yes/no bug judgment plus a line number, with optional free text logged for qualitative analysis only (identify bug); a compliant/not-compliant choice (check specification); or a choice among candidate implementations (select implementation). The recommendation is mapped into the same space, so "final answer = recommendation" is an exact comparison between structured values.

5.2 Verification Support

For a given task the mechanism is identical in all conditions; only the decision to display it differs. It is execution-based and task-specific: execution output or a trace (predict output); predefined tests with failed cases or a counterexample (identify bug); specification-based tests with expected and actual results (check specification); and candidate implementations run against a predefined test suite (select implementation). The system never states whether the recommendation is correct; it may display only `Input: [-5,-2,-8] / Expected: -2 / Actual: 0`, and the participant must interpret the evidence and decide.

5.3 Adaptive Verification Policy

The policy inputs only the participant's initial confidence $c \in [0, 100]$ and displays verification only when $c < \tau$; the threshold τ is fixed before the main experiment—specified *a priori* or set in a separate pilot—and is never adjusted using main-experiment data. The policy never receives the recommendation's correctness and cannot identify errors directly; correctness is used only for evaluation. Task difficulty and type are not policy inputs, only exploratory factors, isolating the primary research variable: whether initial confidence alone can gate verification support. As a premise check, the association between initial confidence and initial answer correctness will be examined in the collected data: low confidence imperfectly predicting erroneous answers supports the threshold as a meaningful trigger, while a weak association is reported as a boundary condition. The adaptive condition is expected to yield a verification rate below 100%; the actual rate is measured empirically, not tuned.

5.4 System Architecture

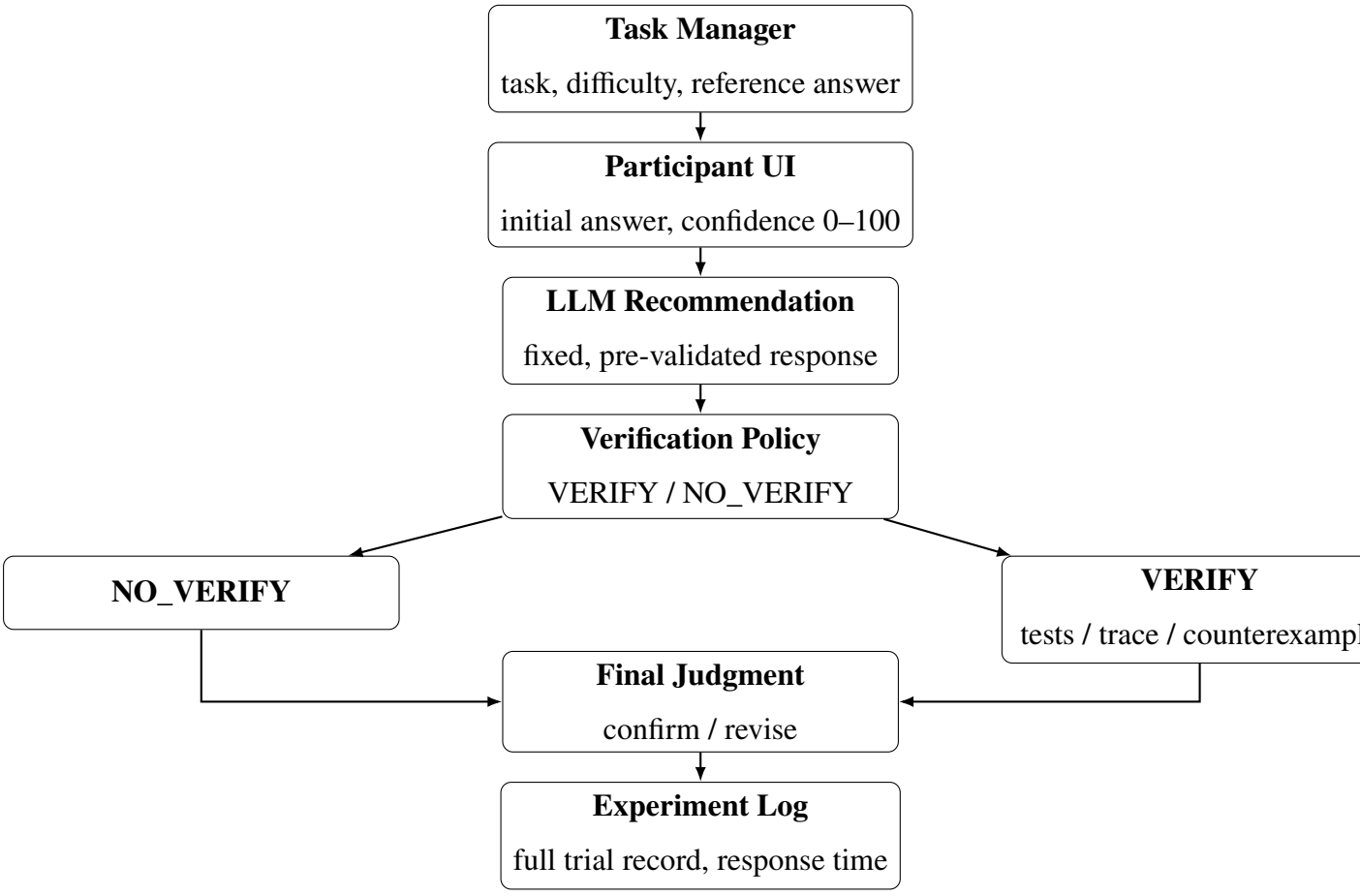


Fig. 5.1. System architecture of the experimental prototype.

The **Task Manager** supplies the task, difficulty, reference answer, and verification data; the **Participant UI** collects the initial answer and confidence; the **LLM Recommendation** component serves the fixed, pre-validated recommendation without revealing its correctness label; the **Verification Policy** decides whether support is displayed (never, always, or by the confidence rule, depending on condition); **Verification Support** presents independent evidence without judging the recommendation; the participant gives a **final judgment** (confirm or revise); and the **Experiment Log** records the complete trial for retrospective analysis of reliance.

5.5 Experimental Design and Procedure

Three conditions are compared: *no verification* (recommendation only), *static verification* (always shown), and *adaptive verification* (shown only when the confidence rule triggers). The mechanism is constant across conditions; the display policy is the manipulated factor. Each participant completes all three conditions—18 trials, six per condition (six no-verification, six static, six adaptive). Each task appears in one condition per participant but in all conditions across participants via counterbalanced assignment, generated before the experiment and balanced for task type, difficulty, and recommendation correctness (exact four-way type balance is impossible, so types are distributed as evenly as possible). Trial order is randomised per participant, avoiding fixed blocks and reducing order, learning, and fatigue effects.

Each trial runs: task $\rightarrow$ initial answer $\rightarrow$ confidence (0–100) $\rightarrow$ fixed LLM recommendation $\rightarrow$ condition-specific verification (evidence examined if displayed) $\rightarrow$ final judgment (confirm or revise). The log records the task identifier, type, and difficulty; initial answer and confidence; recommendation and its correctness; condition and policy decision; verification availability and interaction; final answer; whether the answer changed; and response time.

5.6 Evaluation Metrics

Accuracy, verification rate (VR), and decision-change rate are the proportions of trials with a correct final answer, with displayed verification, and with a changed answer; for static verification VR $= 1$ (subject to exclusion rules), while adaptively VR follows from the confidence distribution and τ. The reliance met-

rics are:

$$\begin{aligned}
\text{OverReliance} &= \frac{N(\text{LLM}_{\text{incorrect}} \wedge \text{Final}{=}\text{LLM})}{N(\text{LLM}_{\text{incorrect}})}, \\
\text{UnderReliance} &= \frac{N(\text{LLM}_{\text{correct}} \wedge \text{Final}{\neq}\text{LLM})}{N(\text{LLM}_{\text{correct}})}, \\
\text{AppropriateReliance} &= \frac{N(\text{LLM}_{\text{correct}} \wedge \text{Final}{=}\text{LLM}) + N(\text{LLM}_{\text{incorrect}} \wedge \text{Final}{\neq}\text{LLM})}{N_{\text{trials}}}.
\end{aligned}$$

Answers are compared as structured values in the closed response space (Section 5.1); a trial counts as changed if any part of the answer differs, and changes are classified where possible as *towards* or *away from* the recommendation, separating acceptance-driven from rejection-driven changes.

Verification cost—the effort required to establish whether an LLM recommendation is correct—is *latent*: it cannot be observed directly when a participant stops early, errs, or does not establish correctness. The study therefore reports verification *burden* and *success* as separate operational measures, all interpreted relative to the fixed experimental protocol (task set, fixed recommendations, verification mechanism, participant population, interface, time budget, exclusion rules) rather than as intrinsic properties of the LLM alone.

Let x denote the task, $\hat{y}$ the fixed LLM recommendation for x, and $g(x, \hat{y}) \in \{\text{correct}, \text{incorrect}\}$ its pre-determined correctness relative to the reference answer. Within a trial, the participant acts as the verifier v, equipped with the interface and with the task-specific verification evidence whenever the experimental condition makes it available. The final structured answer is read as an implicit verdict on $\hat{y}$: accept if it equals $\hat{y}$, reject otherwise (including a change to a third value). Let B denote the per-trial verification budget fixed as part of the protocol (the maximum time allowed for the verification episode, measured from the moment the verification evidence is displayed to the moment the final judgment

is recorded), and let $S_B(x, \hat{y}; v) \in \{0, 1\}$ be the indicator that verifier v reaches a correct verdict on $(x, \hat{y})$ within B:

$$S_B(x, \hat{y}; v) = 1 \iff \big(g(x, \hat{y}) = \text{correct} \wedge \text{Final} = \hat{y}\big) \vee \big(g(x, \hat{y}) = \text{incorrect} \wedge \text{Final} \neq \hat{y}\big),$$

and $S_B(x, \hat{y}; v) = 0$ otherwise, including when no final verdict is recorded within B. Thus a rejection followed by a replacement answer that is itself incorrect still counts as a correct verdict: the recommendation was detected as incorrect, and answer-level quality is tracked separately by final accuracy. The verification success rate (VSR) is

$$\text{VSR} = \frac{N(S_B = 1)}{N_{\text{trials with verification available}}},$$

i.e., trial-level appropriate reliance among trials in which verification was available, restricted to the budget B. VSR captures whether the verdict was reached within the budget, but not that verification *caused* it (a correct recommendation may simply be accepted); it is therefore read jointly with the reliance metrics.

Expected artefacts. The thesis will deliver a validated benchmark (18 tasks with type, difficulty, reference answer, fixed recommendation, correctness label, and verification data), an experimental web application, a pre-validated recommendation dataset, an execution-based verification module, a configurable rule-based policy module, a structured experiment logger, and analysis scripts for all metrics and the confidence–correctness association.

6 Detailed Work Plan and Schedule

The work is planned from October 2026 to mid-May 2027, with the main objective of submitting a research paper by 15 May 2027; the thesis is then extended with additional documentation, analysis, and discussion.

TABLE 6.1
Work plan and schedule

Phase and activities	Period	Deliverable / milestone
1. Literature review and research design: systematic study of appropriate reliance, verification cost, human–AI interaction, confidence-aware support, and LLM assistance in programming tasks	October 2026	Annotated bibliography; refined research gap and research questions
2. Finalisation of the experimental design: operational definitions, hypotheses, metrics, confidence-based policy, verification-cost measures, and statistical analysis plan	1–15 November 2026	Frozen research protocol and analysis plan (M1)
3. Benchmark construction: selection of 18 introductory Python tasks, task categorisation, difficulty assessment, reference-answer validation, and preparation of task-specific verification evidence	15–30 November 2026	Validated programming benchmark (M2)

Phase and activities	Period	Deliverable / milestone
4. Prototype implementation: participant interface, task manager, fixed LLM recommendation module, verification module, adaptive policy, counterbalancing procedure, and experiment logger	December 2026	Functional experimental prototype (M3)
5. Pilot study and design revision: pilot testing of task clarity, interface usability, response-time logging, verification evidence, and confidence elicitation; freezing of the confidence threshold and other policy parameters	January 2027	Pilot report and frozen experimental configuration (M4)
6. Main experiment: participant recruitment, counterbalanced task assignment, data collection, monitoring of exclusions, and quality control	Late January–February 2027	Complete and anonymised trial dataset (M5)
7. Data analysis: data cleaning, descriptive statistics, confidence–correctness analysis, accuracy and reliance metrics, verification cost, response-time analysis, and comparison of static and adaptive verification	February–March 2027	Reproducible analysis scripts and preliminary results (M6)

Phase and activities	Period	Deliverable / milestone
8. Paper preparation: writing the methods, results, discussion, limitations, and conclusion; preparation of tables and figures; internal revision and proofreading	March–April 2027	Complete paper draft and revised manuscript (M7)
9. Paper submission: final revision, formatting according to the target venue, reference checking, preparation of supplementary materials, and submission	By 15 May 2027	Submitted research paper (M8)
10. Thesis continuation and integration: expansion of the paper into the thesis, documentation of the software artefacts, extended analysis, and preparation of the final thesis version	After 15 May 2027	Extended thesis manuscript and final project documentation

Risks and contingencies. The main risks are delays in participant recruitment, problems with the verification module, and insufficient pilot data for threshold selection. Recruitment will begin during the pilot phase and may continue in parallel with prototype validation; if the pilot is inconclusive, the threshold will be specified *a priori* and frozen before the main experiment; if the experiment overruns, the paper will report the preregistered analysis with explicit limitations rather than revising the research questions after observing the data.

7 References

[1] Q. Ma, T. Wu, and K. R. Koedinger, "Human-human pair programming versus human-AI pair programming," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, Association for Computing Machinery, 2023. DOI: 10.1145/3544548.3581225.

[2] D. Cambaz, T. Sönmez, *et al.*, "Use of AI-driven code generation models in teaching and learning programming: A systematic literature review," *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 2*, 2024. DOI: 10.1145/3626252.3630958.

[3] F. Poursabzi-Sangdeh, D. G. Goldstein, J. M. Hofman, J. Wortman Vaughan, and H. Wallach, "Manipulating and measuring model interpretability," *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 2021. DOI: 10.1145/3411764.3445315.

[4] H. Vasconcelos, M. Jörke, M. Grunde-McLaughlin, T. Gerstenberg, M. S. Bernstein, and R. Krishna, "Explanations can reduce overreliance on AI systems during decision-making," *Proceedings of the ACM on Human-Computer Interaction*, vol. 7, no. CSCW1, 2023. DOI: 10.1145/3579605.

[5] Z. Buçinca, M. B. Malaya, and K. Z. Gajos, "To trust or to think: Cognitive forcing functions can reduce overreliance on AI in AI-assisted decision-making," *Proceedings of the ACM on Human-Computer Interaction*, vol. 5, no. CSCW1, 2021. DOI: 10.1145/3449287.

[6] S. Ma, X. Wang, Y. Lei, C. Shi, M. Yin, and X. Ma, *"are you really sure?" understanding the effects of human self-confidence calibration in ai-assisted*

decision making, 2024. arXiv: 2403.09552 [cs.HC]. [Online]. Available: https://arxiv.org/abs/2403.09552.

[7] M. Kraus *et al.*, "To rely or not to rely? evaluating interventions for appropriate reliance on large language models," *arXiv preprint arXiv:2412.15584*, 2024. [Online]. Available: https://arxiv.org/abs/2412.15584.

[8] V. Crescitelli, G. Immediato, F. Persia, and S. Costantini, *Ai evaluation should measure verification cost, not correctness alone*, 2026. arXiv: 2608.08709 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2608.08709.

[9] Y. Xie *et al.*, "Adjust for trust: Mitigating trust-induced inappropriate reliance on AI assistance," *arXiv preprint arXiv:2502.13321*, 2025. [Online]. Available: https://arxiv.org/abs/2502.13321.